

main.py

```
1 import os
2 import json
3 from datetime import datetime
4 import torch
5 from transformers import AutoModelForCausalLM, AutoTokenizer
6 from tqdm import tqdm
7 from colorama import Fore, Style
8 import colorama
9 import re
10
11 colorama.init()
12
13 def sanitize_response(text):
14     """Remove Markdown strikethrough and HTML <del> tags from model output."""
15     text = re.sub(r'~~', '', text)
16     text = re.sub(r'</?del>', '', text)
17     return text
18
19 class AIAssistant:
20     def __init__(self, name="SRUSTI_AI_ASSISTANT"):
21         self.name = name
22         self.feedback_data = []
23         self.feedback_file = "feedback_data.json"
24         self.load_feedback()
25
26         # Load model and tokenizer with error handling
27         print(f"{Fore.CYAN}Loading model and tokenizer... (this may take a moment)
28 {Style.RESET_ALL}")
29         self.model_name = "TinyLlama/TinyLlama-1.1B-Chat-v1.0" # A smaller model
30
31         try:
32             self.tokenizer = AutoTokenizer.from_pretrained(self.model_name)
33
34             # Show progress bar during model loading
35             with tqdm(total=100, desc="Loading model") as pbar:
36                 self.model = AutoModelForCausalLM.from_pretrained(
37                     self.model_name,
38                     torch_dtype=torch.float16,
39                     device_map="auto"
40                 )
41                 pbar.update(100)
42
43             print(f"{Fore.GREEN}Model loaded successfully: {self.model_name}
44 {Style.RESET_ALL}")
45         except Exception as e:
46             print(f"{Fore.RED}Error loading model: {e}{Style.RESET_ALL}")
47             print(f"{Fore.YELLOW}Falling back to CPU mode with reduced
48 performance{Style.RESET_ALL}")
49
50         try:
51             self.tokenizer = AutoTokenizer.from_pretrained(self.model_name)
52             self.model = AutoModelForCausalLM.from_pretrained(
53                 self.model_name,
```

```

50         torch_dtype=torch.float32, # Use float32 as fallback
51         device_map="cpu"
52     )
53     except Exception as e2:
54         print(f"{Fore.RED}Critical error: {e2}. Cannot continue.
{Style.RESET_ALL}")
55         raise e2
56
57     # Store conversation history
58     self.conversation_history = []
59
60     def load_feedback(self):
61         """Load previous feedback if available"""
62         try:
63             if os.path.exists(self.feedback_file):
64                 with open(self.feedback_file, 'r') as f:
65                     self.feedback_data = json.load(f)
66                     print(f"{Fore.GREEN}Loaded {len(self.feedback_data)} feedback
entries{Style.RESET_ALL}")
67             except Exception as e:
68                 print(f"{Fore.RED}Error loading feedback data: {e}{Style.RESET_ALL}")
69
70     def save_feedback(self):
71         """Save feedback to file"""
72         try:
73             with open(self.feedback_file, 'w') as f:
74                 json.dump(self.feedback_data, f, indent=2)
75                 print(f"{Fore.GREEN}Feedback saved successfully{Style.RESET_ALL}")
76             except Exception as e:
77                 print(f"{Fore.RED}Error saving feedback data: {e}{Style.RESET_ALL}")
78
79     def display_menu(self):
80         """Display the main menu"""
81         print(f"\n{Fore.CYAN}{'='*60}{Style.RESET_ALL}")
82         print(f"{Fore.YELLOW>Welcome to {self.name}!{Style.RESET_ALL}")
83         print(f"{Fore.CYAN}{'='*60}{Style.RESET_ALL}")
84         print(f"{Fore.WHITE}1. Answer Questions")
85         print(f"2. Summarize Text")
86         print(f"3. Generate Creative Content")
87         print(f"4. View Conversation History")
88         print(f"5. Exit{Style.RESET_ALL}")
89         print(f"{Fore.CYAN}{'='*60}{Style.RESET_ALL}")
90
91     def get_ai_response(self, prompt, function_type):
92         """Get response from the model with improved error handling and progress
tracking"""
93         # Set different system prompts based on function type
94         if function_type == "question":
95             system_prompt = "You are a helpful assistant providing accurate, factual
answers. Be concise but thorough."
96         elif function_type == "summary":
97             system_prompt = "You are a summarization expert. Create concise summaries that
capture key points while reducing length by 70%."
98         elif function_type == "creative":

```

```

99         system_prompt = "You are a creative writing assistant with excellent
imagination and writing skills."
100     else:
101         system_prompt = "You are a helpful assistant."
102
103     # Format the prompt for the model
104     formatted_prompt = f"
<|system|>\n{system_prompt}\n<|user|>\n{prompt}\n<|assistant|>"
105
106     try:
107         # Check if input is too long and truncate if necessary
108         max_input_length = self.tokenizer.model_max_length - 512 # Reserve tokens for
the response
109         input_ids = self.tokenizer(prompt, return_tensors="pt").input_ids
110
111         if input_ids.shape[1] > max_input_length:
112             print(f"{Fore.YELLOW}Input is too long. Truncating to fit model context
window.{Style.RESET_ALL}")
113             truncated_prompt = self.tokenizer.decode(input_ids[0, :max_input_length])
114             formatted_prompt = f"
<|system|>\n{system_prompt}\n<|user|>\n{truncated_prompt} [truncated...]\n<|assistant|>"
115
116         # Tokenize and generate
117         inputs = self.tokenizer(formatted_prompt,
return_tensors="pt").to(self.model.device)
118
119         # Set generation parameters
120         gen_kwargs = {
121             "max_new_tokens": 512,
122             "temperature": 0.7 if function_type == "creative" else 0.3,
123             "top_p": 0.9,
124             "do_sample": True
125         }
126
127         # Generate response with progress bar
128         print(f"{Fore.CYAN}Generating response...{Style.RESET_ALL}")
129         with torch.no_grad():
130             with tqdm(total=100, desc="Generating") as pbar:
131                 output_sequences = self.model.generate(**inputs, **gen_kwargs)
132                 pbar.update(100)
133
134         # Decode and clean up the response
135         response = self.tokenizer.decode(output_sequences[0],
skip_special_tokens=False)
136         response = response.replace(formatted_prompt, "").split("<|end|>")[0].strip()
137         # Sanitize here!
138         response = sanitize_response(response)
139
140         # Store in conversation history
141         self.conversation_history.append({
142             "timestamp": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
143             "type": function_type,
144             "prompt": prompt,
145             "response": response

```

```

146         })
147
148         return response
149     except Exception as e:
150         error_msg = f"Error generating response: {str(e)}"
151         print(f"{Fore.RED}{error_msg}{Style.RESET_ALL}")
152         return error_msg
153
154     def answer_question(self):
155         """Function to answer factual questions"""
156         print(f"\n{Fore.CYAN}=== Question Answering Mode ==={Style.RESET_ALL}")
157         question = input(f"{Fore.YELLOW}Your question: {Style.RESET_ALL}")
158
159         # Three different prompt variations
160         prompt_variations = [
161             f"Answer this question accurately and concisely: {question}",
162             f"Provide factual information about: {question}",
163             f"Explain in detail: {question}"
164         ]
165
166         print(f"\n{Fore.CYAN}Choose a prompt style:{Style.RESET_ALL}")
167         for i, prompt in enumerate(prompt_variations, 1):
168             print(f"{i}. {prompt}")
169
170         try:
171             choice = int(input(f"{Fore.YELLOW}Select prompt style (1-3) or press Enter for
default: {Style.RESET_ALL}") or "1")
172             if choice < 1 or choice > 3:
173                 choice = 1
174         except ValueError:
175             choice = 1
176
177         selected_prompt = prompt_variations[choice-1]
178         print(f"\n{Fore.GREEN}Using prompt: {selected_prompt}{Style.RESET_ALL}")
179
180         print(f"\n{Fore.CYAN}Generating response...{Style.RESET_ALL}")
181         response = self.get_ai_response(selected_prompt, "question")
182
183         print(f"\n{Fore.GREEN}--- Answer ---{Style.RESET_ALL}")
184         print(f"{Fore.WHITE}{response}{Style.RESET_ALL}")
185
186         self.collect_feedback("question", selected_prompt, response)
187         return response
188
189     def summarize_text(self):
190         """Function to summarize text"""
191         print(f"\n{Fore.CYAN}=== Text Summarization Mode ==={Style.RESET_ALL}")
192         print(f"{Fore.YELLOW}Enter or paste the text you want to summarize:
{Style.RESET_ALL}")
193         text = input(f"{Fore.YELLOW}Text: {Style.RESET_ALL}")
194
195         # Check if text is too long and warn user
196         token_count = len(self.tokenizer.encode(text))

```

```

197         max_tokens = self.tokenizer.model_max_length - 256 # Reserve tokens for prompt
and response
198
199         if token_count > max_tokens:
200             print(f"{Fore.RED}Warning: Your text is {token_count} tokens, which exceeds
the model's capacity.{Style.RESET_ALL}")
201             print(f"{Fore.YELLOW}The text will be truncated to {max_tokens} tokens.
{Style.RESET_ALL}")
202             text = self.tokenizer.decode(self.tokenizer.encode(text)[:max_tokens])
203
204         # Three different prompt variations
205         prompt_variations = [
206             f"Summarize the following text concisely:\n\n{text}",
207             f"Extract and list the key points from this text:\n\n{text}",
208             f"Provide a brief overview of the main ideas in this text:\n\n{text}"
209         ]
210
211         print(f"\n{Fore.CYAN}Choose a summarization style:{Style.RESET_ALL}")
212         for i, prompt in enumerate(prompt_variations, 1):
213             print(f"{i}. Style {i}")
214
215         try:
216             choice = int(input(f"{Fore.YELLOW}Select style (1-3) or press Enter for
default: {Style.RESET_ALL}") or "1")
217             if choice < 1 or choice > 3:
218                 choice = 1
219         except ValueError:
220             choice = 1
221
222         selected_prompt = prompt_variations[choice-1]
223
224         print(f"\n{Fore.CYAN}Generating summary...{Style.RESET_ALL}")
225         response = self.get_ai_response(selected_prompt, "summary")
226
227         print(f"\n{Fore.GREEN}--- Summary ---{Style.RESET_ALL}")
228         print(f"{Fore.WHITE}{response}{Style.RESET_ALL}")
229
230         self.collect_feedback("summary", selected_prompt, response)
231         return response
232
233     def generate_creative_content(self):
234         """Function to generate creative content"""
235         print(f"\n{Fore.CYAN}=== Creative Content Generation ==={Style.RESET_ALL}")
236         content_type = input(f"{Fore.YELLOW}What type of creative content would you like?
(story/poem/essay): {Style.RESET_ALL}")
237         topic = input(f"{Fore.YELLOW}Enter a topic or theme for your {content_type}:
{Style.RESET_ALL}")
238
239         # Three different prompt variations
240         prompt_variations = [
241             f"Write a {content_type} about {topic}.",
242             f"Create an original {content_type} with the theme: {topic}. Be imaginative
and engaging.",
243             f"Generate a creative {content_type} that explores {topic} in a unique and
interesting way."

```

```

244     ]
245
246     print(f"\n{Fore.CYAN}Choose a creativity style:{Style.RESET_ALL}")
247     for i, prompt in enumerate(prompt_variations, 1):
248         print(f"{i}. {prompt}")
249
250     try:
251         choice = int(input(f"{Fore.YELLOW}Select style (1-3) or press Enter for
default: {Style.RESET_ALL}") or "1")
252         if choice < 1 or choice > 3:
253             choice = 1
254     except ValueError:
255         choice = 1
256
257     selected_prompt = prompt_variations[choice-1]
258     print(f"\n{Fore.GREEN}Using prompt: {selected_prompt}{Style.RESET_ALL}")
259
260     print(f"\n{Fore.CYAN}Generating creative content...{Style.RESET_ALL}")
261     response = self.get_ai_response(selected_prompt, "creative")
262
263     print(f"\n{Fore.GREEN}--- Your {content_type.capitalize()} ---{Style.RESET_ALL}")
264     print(f"{Fore.WHITE}{response}{Style.RESET_ALL}")
265
266     self.collect_feedback("creative", selected_prompt, response)
267     return response
268
269 def view_history(self):
270     """View conversation history"""
271     if not self.conversation_history:
272         print(f"\n{Fore.YELLOW}No conversation history available.{Style.RESET_ALL}")
273         return
274
275     print(f"\n{Fore.CYAN}=== Conversation History ==={Style.RESET_ALL}")
276     for i, entry in enumerate(self.conversation_history, 1):
277         print(f"\n{Fore.GREEN}[[{i}]] {entry['timestamp']} - {entry['type'].upper()}
{Style.RESET_ALL}")
278         print(f"{Fore.YELLOW}Prompt: {entry['prompt']}{Style.RESET_ALL}")
279         print(f"{Fore.WHITE}Response: {entry['response']}{Style.RESET_ALL}")
280
281     # Option to save history to file
282     save_option = input(f"\n{Fore.YELLOW}Save history to file? (y/n):
{Style.RESET_ALL}")
283     if save_option.lower() == 'y':
284         try:
285             history_file = f"conversation_history-
_{datetime.now().strftime('%Y%m%d_%H%M%S')}.json"
286             with open(history_file, 'w') as f:
287                 json.dump(self.conversation_history, f, indent=2)
288             print(f"{Fore.GREEN}History saved to {history_file}{Style.RESET_ALL}")
289         except Exception as e:
290             print(f"{Fore.RED}Error saving history: {e}{Style.RESET_ALL}")
291
292 def collect_feedback(self, function_type, prompt, response):
293     """Collect user feedback on the AI's response"""

```

```

294     print(f"\n{Fore.CYAN}--- Feedback ---{Style.RESET_ALL}")
295     print(f"{Fore.YELLOW}How would you rate this response? (1-5, where 5 is excellent)
{Style.RESET_ALL}")
296     try:
297         rating = int(input(f"{Fore.YELLOW}Rating: {Style.RESET_ALL}") or "0")
298         if rating < 1 or rating > 5:
299             print(f"{Fore.YELLOW}Invalid rating. Skipping feedback.{Style.RESET_ALL}")
300             return
301     except ValueError:
302         print(f"{Fore.YELLOW}Invalid input. Skipping feedback.{Style.RESET_ALL}")
303         return
304
305     comments = input(f"{Fore.YELLOW}Any additional comments? (optional):
{Style.RESET_ALL}")
306
307     # Store feedback
308     feedback_entry = {
309         "timestamp": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
310         "function_type": function_type,
311         "prompt": prompt,
312         "response": response,
313         "rating": rating,
314         "comments": comments
315     }
316
317     self.feedback_data.append(feedback_entry)
318     self.save_feedback()
319     print(f"{Fore.GREEN}Thank you for your feedback!{Style.RESET_ALL}")
320
321     def run(self):
322         """Main execution loop for the assistant"""
323         # Display device information
324         device_info = "CUDA" if torch.cuda.is_available() else "CPU"
325         print(f"{Fore.CYAN}Running on: {device_info}{Style.RESET_ALL}")
326
327         while True:
328             self.display_menu()
329             choice = input(f"\n{Fore.YELLOW}Enter your choice (1-5): {Style.RESET_ALL}")
330
331             if choice == '1':
332                 self.answer_question()
333             elif choice == '2':
334                 self.summarize_text()
335             elif choice == '3':
336                 self.generate_creative_content()
337             elif choice == '4':
338                 self.view_history()
339             elif choice == '5':
340                 print(f"\n{Fore.GREEN}Thank you for using {self.name}. Goodbye!
{Style.RESET_ALL}")
341                 break
342             else:
343                 print(f"\n{Fore.RED}Invalid choice. Please try again.{Style.RESET_ALL}")
344

```

```

345         input(f"\n{Fore.CYAN}Press Enter to continue...{Style.RESET_ALL}")
346
347     def save_conversation_history(self, filename=None):
348         """Save conversation history to a file"""
349         if not filename:
350             filename = f"conversation_history-
_{datetime.now().strftime('%Y%m%d_%H%M%S')}.json"
351
352         try:
353             with open(filename, 'w') as f:
354                 json.dump(self.conversation_history, f, indent=2)
355                 print(f"{Fore.GREEN}Conversation history saved to {filename}
{Style.RESET_ALL}")
356             return True
357         except Exception as e:
358             print(f"{Fore.RED}Error saving conversation history: {e}{Style.RESET_ALL}")
359             return False
360
361     def load_conversation_history(self, filename):
362         """Load conversation history from a file"""
363         try:
364             with open(filename, 'r') as f:
365                 self.conversation_history = json.load(f)
366                 print(f"{Fore.GREEN}Loaded {len(self.conversation_history)} conversation
entries from {filename}{Style.RESET_ALL}")
367             return True
368         except Exception as e:
369             print(f"{Fore.RED}Error loading conversation history: {e}{Style.RESET_ALL}")
370             return False
371
372
373 def main():
374     """Main function to run the AI Assistant"""
375     print(f"{Fore.CYAN}{' '*60}{Style.RESET_ALL}")
376     print(f"{Fore.YELLOW}Initializing AI Assistant...{Style.RESET_ALL}")
377     print(f"{Fore.CYAN}{' '*60}{Style.RESET_ALL}")
378
379     # Check for CUDA availability
380     if torch.cuda.is_available():
381         print(f"{Fore.GREEN}CUDA is available. Using GPU acceleration.{Style.RESET_ALL}")
382         print(f"{Fore.GREEN}GPU: {torch.cuda.get_device_name(0)}{Style.RESET_ALL}")
383     else:
384         print(f"{Fore.YELLOW}CUDA is not available. Using CPU mode (slower).
{Style.RESET_ALL}")
385
386     try:
387         assistant = AIAssistant()
388         assistant.run()
389     except KeyboardInterrupt:
390         print(f"\n{Fore.YELLOW}Program interrupted by user.{Style.RESET_ALL}")
391     except Exception as e:
392         print(f"\n{Fore.RED}An error occurred: {e}{Style.RESET_ALL}")
393     finally:
394         # Clean up resources

```

```
395         print(f"{Fore.CYAN}Cleaning up resources...{Style.RESET_ALL}")
396         colorama.deinit()
397
398
399 if __name__ == "__main__":
400     main()
401
```

app.py

```
1 import os
2 import json
3 from datetime import datetime
4 import torch
5 from flask import Flask, render_template, request, jsonify, session, Response
6 import threading
7 import uuid
8
9 # Import the AIAssistant class from main.py
10 from main import AIAssistant
11
12 app = Flask(__name__)
13 app.secret_key = os.urandom(24) # For session management
14
15 # Global variable to store the model and assistant
16 global_assistant = None
17 model_loading_status = {"status": "not_started", "progress": 0, "message": ""}
18
19 def load_model_in_background():
20     """Load the model in a background thread to avoid blocking the web server"""
21     global global_assistant, model_loading_status
22     try:
23         model_loading_status = {"status": "loading", "progress": 10, "message":
24 "Initializing AI Assistant..."}
25         # Create the AI Assistant instance
26         global_assistant = AIAssistant(name="SRUSTI_AI_ASSISTANT")
27         model_loading_status = {"status": "complete", "progress": 100, "message": "Model
28 loaded successfully!"}
29     except Exception as e:
30         model_loading_status = {"status": "error", "progress": 0, "message": f"Error
31 loading model: {str(e)}"}
32         print(f"Error loading model: {e}")
33
34 # Start loading the model in the background when the app starts
35 threading.Thread(target=load_model_in_background).start()
36
37 @app.route('/')
38 def index():
39     """Render the main page"""
40     # Generate a unique session ID if not present
41     if 'user_id' not in session:
42         session['user_id'] = str(uuid.uuid4())
43         session['conversation_history'] = []
44     return render_template('index.html')
45
46 @app.route('/model_status')
47 def model_status():
48     """Return the current status of model loading"""
49     global model_loading_status
50     return jsonify(model_loading_status)
51
52 @app.route('/generate', methods=['POST'])
```

```

50 def generate():
51     """Generate a response based on the input"""
52     global global_assistant
53     if global_assistant is None:
54         return jsonify({
55             "success": False,
56             "error": "Model is still loading. Please wait.",
57             "model_status": model_loading_status
58         })
59     data = request.json
60     prompt = data.get('prompt', '')
61     function_type = data.get('function_type', 'question') # question, summary, creative
62     if not prompt:
63         return jsonify({"success": False, "error": "Prompt cannot be empty"})
64     try:
65         # Use the appropriate method from AIAssistant based on function_type
66         if function_type == "question":
67             response = global_assistant.get_ai_response(prompt, "question")
68         elif function_type == "summary":
69             response = global_assistant.get_ai_response(prompt, "summary")
70         elif function_type == "creative":
71             response = global_assistant.get_ai_response(prompt, "creative")
72         else:
73             response = global_assistant.get_ai_response(prompt, "question")
74         # Store in session conversation history
75         if 'conversation_history' in session:
76             session['conversation_history'].append({
77                 "timestamp": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
78                 "type": function_type,
79                 "prompt": prompt,
80                 "response": response
81             })
82             session.modified = True
83         return jsonify({
84             "success": True,
85             "response": response,
86             "function_type": function_type
87         })
88     except Exception as e:
89         print(f"Error generating response: {e}")
90         return jsonify({
91             "success": False,
92             "error": f"Error generating response: {str(e)}"
93         })
94
95 @app.route('/feedback', methods=['POST'])
96 def feedback():
97     """Collect feedback on a response"""
98     global global_assistant
99     if global_assistant is None:
100         return jsonify({"success": False, "error": "AI Assistant not initialized"})
101     data = request.json
102     function_type = data.get('function_type', '')
103     prompt = data.get('prompt', '')

```

```

104     response = data.get('response', '')
105     rating = data.get('rating', 0)
106     comments = data.get('comments', '')
107     # Create feedback entry
108     feedback_entry = {
109         "timestamp": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
110         "function_type": function_type,
111         "prompt": prompt,
112         "response": response,
113         "rating": rating,
114         "comments": comments
115     }
116     # Add to assistant's feedback data
117     global_assistant.feedback_data.append(feedback_entry)
118     global_assistant.save_feedback()
119     return jsonify({
120         "success": True,
121         "message": "Thank you for your feedback!"
122     })
123
124 @app.route('/history', methods=['GET'])
125 def history():
126     """Get conversation history from the session"""
127     if 'conversation_history' in session:
128         return jsonify({
129             "success": True,
130             "history": session['conversation_history']
131         })
132     else:
133         return jsonify({
134             "success": False,
135             "error": "No conversation history found"
136         })
137
138 @app.route('/clear_history', methods=['POST'])
139 def clear_history():
140     """Clear the conversation history from the session"""
141     if 'conversation_history' in session:
142         session['conversation_history'] = []
143         session.modified = True
144         return jsonify({
145             "success": True,
146             "message": "Conversation history cleared"
147         })
148     else:
149         return jsonify({
150             "success": False,
151             "error": "No conversation history found"
152         })
153
154 @app.route('/download_history', methods=['GET'])
155 def download_history():
156     """Generate a downloadable JSON file of the conversation history"""
157     if 'conversation_history' not in session or not session['conversation_history']:

```

```

158         return jsonify({
159             "success": False,
160             "error": "No conversation history to download"
161         })
162     history_data = json.dumps(session['conversation_history'], indent=2)
163     return Response(
164         history_data,
165         mimetype='application/json',
166         headers={
167             'Content-Disposition': f'attachment;filename=conversation_history-
_{datetime.now().strftime("%Y%m%d_%H%M%S")}.json'
168         }
169     )
170
171 @app.route('/health', methods=['GET'])
172 def health_check():
173     """Simple health check endpoint"""
174     return jsonify({
175         "status": "ok",
176         "timestamp": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
177         "model_loaded": global_assistant is not None,
178         "model_status": model_loading_status
179     })
180
181 @app.errorhandler(404)
182 def page_not_found(e):
183     """Handle 404 errors"""
184     return jsonify({
185         "success": False,
186         "error": "The requested resource was not found"
187     }), 404
188
189 @app.errorhandler(500)
190 def server_error(e):
191     """Handle 500 errors"""
192     return jsonify({
193         "success": False,
194         "error": "An internal server error occurred"
195     }), 500
196
197 if __name__ == '__main__':
198     # Check if CUDA is available and print info
199     if torch.cuda.is_available():
200         print(f"CUDA is available. Using GPU: {torch.cuda.get_device_name(0)}")
201     else:
202         print("CUDA is not available. Using CPU mode (slower).")
203     # Create templates directory if it doesn't exist
204     os.makedirs('templates', exist_ok=True)
205     # Check if index.html exists in templates directory
206     if not os.path.exists('templates/index.html'):
207         print("Warning: templates/index.html not found. Please make sure to create this
file.")
208     print("The application will still start, but you may see a 'TemplateNotFound'
error when accessing the web interface.")

```

```
209 |         # Run the Flask app
210 |     print("Starting AI Assistant web server...")
211 |     print("Access the web interface at http://localhost:5000")
212 |     app.run(debug=True, host='0.0.0.0', port=5000)
```

templates\index.html

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>SRUSTI AI Assistant</title>
7   <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
8   <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.0.0/css/all.min.css">
9   <style>
10     body { background-color: #f8f9fa; font-family: 'Segoe UI', Tahoma, Geneva,
Verdana, sans-serif; }
11     .chat-container { height: 70vh; overflow-y: auto; padding: 20px; background-color:
white; border-radius: 10px; box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);}
12     .message { margin-bottom: 15px; padding: 10px 15px; border-radius: 10px; max-
width: 80%; }
13     .user-message { background-color: #007bff; color: white; margin-left: auto; }
14     .assistant-message { background-color: #e9ecef; color: #212529; }
15     .loading-spinner { display: inline-block; width: 2rem; height: 2rem; vertical-
align: text-bottom; border: 0.25em solid currentColor; border-right-color: transparent;
border-radius: 50%; animation: spinner-border .75s linear infinite; }
16     .feedback-stars { color: #ffc107; cursor: pointer; font-size: 1.5rem; }
17     .tab-content { padding: 20px 0; }
18     .model-loading { text-align: center; padding: 50px 0; }
19     pre { background-color: #f8f9fa; padding: 10px; border-radius: 5px; overflow-x:
auto; }
20     code { font-family: 'Courier New', Courier, monospace; }
21     .nav-tabs .nav-link { font-weight: 500; }
22     .nav-tabs .nav-link.active { font-weight: 600; color: #007bff; }
23     /* --- Add this to force-disable strikethrough --- */
24     del, s, strike {
25       text-decoration: none !important;
26     }
27   </style>
28 </head>
29 <body>
30   <div class="container mt-4">
31     <div class="row">
32       <div class="col-12">
33         <h1 class="text-center mb-4">
34           <i class="fas fa-robot text-primary me-2"></i>
35           SRUSTI AI Assistant
36         </h1>
37       </div>
38     </div>
39     <!-- Model Loading Status -->
40     <div id="modelLoading" class="model-loading">
41       <h3>Loading AI Model...</h3>
42       <div class="progress mt-3 mb-3" style="height: 25px;">
43         <div id="modelLoadingProgress" class="progress-bar progress-bar-striped
progress-bar-animated">
```

```

44         role="progressbar" style="width: 0%;" aria-valuenow="0" aria-
valuemin="0" aria-valuemax="100">0%</div>
45     </div>
46     <p id="modelLoadingMessage">Initializing...</p>
47 </div>
48 <!-- Main Interface (hidden until model loads) -->
49 <div id="mainInterface" class="row" style="display: none;">
50     <div class="col-12">
51         <ul class="nav nav-tabs" id="myTab" role="tablist">
52             <li class="nav-item" role="presentation">
53                 <button class="nav-link active" id="chat-tab" data-bs-toggle="tab"
data-bs-target="#chat"
54                     type="button" role="tab" aria-controls="chat" aria-
selected="true">
55                     <i class="fas fa-comment-dots me-2"></i>Chat
56                 </button>
57             </li>
58             <li class="nav-item" role="presentation">
59                 <button class="nav-link" id="summarize-tab" data-bs-toggle="tab"
data-bs-target="#summarize"
60                     type="button" role="tab" aria-controls="summarize" aria-
selected="false">
61                     <i class="fas fa-file-alt me-2"></i>Summarize
62                 </button>
63             </li>
64             <li class="nav-item" role="presentation">
65                 <button class="nav-link" id="creative-tab" data-bs-toggle="tab"
data-bs-target="#creative"
66                     type="button" role="tab" aria-controls="creative" aria-
selected="false">
67                     <i class="fas fa-paint-brush me-2"></i>Creative
68                 </button>
69             </li>
70             <li class="nav-item" role="presentation">
71                 <button class="nav-link" id="history-tab" data-bs-toggle="tab"
data-bs-target="#history"
72                     type="button" role="tab" aria-controls="history" aria-
selected="false">
73                     <i class="fas fa-history me-2"></i>History
74                 </button>
75             </li>
76         </ul>
77         <div class="tab-content" id="myTabContent">
78             <!-- Chat Tab -->
79             <div class="tab-pane fade show active" id="chat" role="tabpanel" aria-
labelledby="chat-tab">
80                 <div class="chat-container mb-3" id="chatMessages">
81                     <div class="text-center text-muted py-5">
82                         <i class="fas fa-comment-dots fa-3x mb-3"></i>
83                         <p>Ask me anything! I'm here to help.</p>
84                     </div>
85                 </div>
86                 <div class="input-group mb-3">
87                     <input type="text" id="questionInput" class="form-control"

```

```

88         placeholder="Type your question here..." aria-
label="Question">
89         <button class="btn btn-primary" type="button"
id="sendQuestionBtn">
90             <i class="fas fa-paper-plane"></i> Send
91         </button>
92     </div>
93 </div>
94 <!-- Summarize Tab -->
95 <div class="tab-pane fade" id="summarize" role="tabpanel" aria-
labelledby="summarize-tab">
96     <div class="mb-3">
97         <label for="textToSummarize" class="form-label">Enter text to
summarize:</label>
98         <textarea class="form-control" id="textToSummarize" rows="10"
placeholder="Paste your text here..."></textarea>
99     </div>
100     <div class="d-grid gap-2">
101         <button class="btn btn-primary" type="button"
id="summarizeBtn">
102             <i class="fas fa-compress-alt"></i> Summarize
103         </button>
104     </div>
105     <div class="mt-4" id="summaryResult" style="display: none;">
106         <h4>Summary:</h4>
107         <div class="card">
108             <div class="card-body" id="summaryText"></div>
109         </div>
110         <div class="mt-3 text-center" id="summaryFeedback"></div>
111     </div>
112 </div>
113 <!-- Creative Tab -->
114 <div class="tab-pane fade" id="creative" role="tabpanel" aria-
labelledby="creative-tab">
115     <div class="row mb-3">
116         <div class="col-md-6">
117             <label for="contentType" class="form-label">Content Type:
</label>
118             <select class="form-select" id="contentType">
119                 <option value="story">Story</option>
120                 <option value="poem">Poem</option>
121                 <option value="essay">Essay</option>
122                 <option value="dialogue">Dialogue</option>
123                 <option value="song">Song Lyrics</option>
124             </select>
125         </div>
126         <div class="col-md-6">
127             <label for="contentTopic" class="form-label">Topic or
Theme:</label>
128             <input type="text" class="form-control" id="contentTopic"
placeholder="Enter a topic or theme...">
129         </div>
130     </div>
131 </div>
132 </div>
133 <div class="d-grid gap-2">

```

```

134         <button class="btn btn-primary" type="button"
id="generateCreativeBtn">
135             <i class="fas fa-magic"></i> Generate Creative Content
136         </button>
137     </div>
138     <div class="mt-4" id="creativeResult" style="display: none;">
139         <h4 id="creativeTitle">Your Creative Content:</h4>
140         <div class="card">
141             <div class="card-body" id="creativeText"></div>
142         </div>
143         <div class="mt-3 text-center" id="creativeFeedback"></div>
144     </div>
145 </div>
146 <!-- History Tab -->
147 <div class="tab-pane fade" id="history" role="tabpanel" aria-
labelledby="history-tab">
148     <div class="d-flex justify-content-end mb-3">
149         <button class="btn btn-outline-secondary me-2"
id="downloadHistoryBtn">
150             <i class="fas fa-download"></i> Download History
151         </button>
152         <button class="btn btn-outline-danger" id="clearHistoryBtn">
153             <i class="fas fa-trash"></i> Clear History
154         </button>
155     </div>
156     <div id="historyContainer">
157         <div class="text-center text-muted py-5" id="emptyHistory">
158             <i class="fas fa-history fa-3x mb-3"></i>
159             <p>No conversation history yet.</p>
160         </div>
161         <div id="historyList"></div>
162     </div>
163 </div>
164 </div>
165 </div>
166 </div>
167 </div>
168 <!-- JavaScript -->
169 <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js">
</script>
170 <script src="https://cdn.jsdelivr.net/npm/marked/marked.min.js"></script>
171 <script>
172     // Disable strikethrough in marked
173     marked.use({
174         renderer: {
175             del(text) { return text; } // just return the text, no <del> tag
176         }
177     });
178
179     // Helper to remove all strikethrough tags and markdown
180     function sanitizeResponse(text) {
181         return text
182             .replace(/~~/g, "")

```

```

183         .replace(/<\/?del>/g, "")
184         .replace(/<\/?s>/g, "")
185         .replace(/<\/?strike>/g, "");
186     }
187
188     document.addEventListener('DOMContentLoaded', function() {
189         checkModelStatus();
190         document.getElementById('sendQuestionBtn').addEventListener('click',
sendQuestion);
191         document.getElementById('questionInput').addEventListener('keypress',
function(e) {
192             if (e.key === 'Enter') sendQuestion();
193         });
194         document.getElementById('summarizeBtn').addEventListener('click',
summarizeText);
195         document.getElementById('generateCreativeBtn').addEventListener('click',
generateCreative);
196         document.getElementById('clearHistoryBtn').addEventListener('click',
clearHistory);
197         document.getElementById('downloadHistoryBtn').addEventListener('click',
downloadHistory);
198         document.getElementById('history-tab').addEventListener('shown.bs.tab',
loadHistory);
199     });
200
201     function checkModelStatus() {
202         fetch('/model_status')
203         .then(response => response.json())
204         .then(data => {
205             const progressBar = document.getElementById('modelLoadingProgress');
206             const message = document.getElementById('modelLoadingMessage');
207             progressBar.style.width = data.progress + '%';
208             progressBar.textContent = data.progress + '%';
209             message.textContent = data.message;
210             if (data.status === "complete") {
211                 document.getElementById('modelLoading').style.display = 'none';
212                 document.getElementById('mainInterface').style.display = 'block';
213             } else if (data.status === "error") {
214                 message.textContent = "Error loading model: " + data.message;
215                 progressBar.classList.add('bg-danger');
216             } else {
217                 setTimeout(checkModelStatus, 1000);
218             }
219         })
220         .catch(error => {
221             console.error('Error checking model status:', error);
222             document.getElementById('modelLoadingMessage').textContent = 'Error
checking model status. Please refresh.';
223             document.getElementById('modelLoadingProgress').classList.add('bg-
danger');
224         });
225     }
226
227     function sendQuestion() {
228         const questionInput = document.getElementById('questionInput');

```

```

229     const question = questionInput.value.trim();
230     if (!question) return;
231     addMessageToChat('user', question);
232     questionInput.value = '';
233     const loadingId = addLoadingMessage();
234     fetch('/generate', {
235         method: 'POST',
236         headers: { 'Content-Type': 'application/json' },
237         body: JSON.stringify({ prompt: question, function_type: 'question' })
238     })
239     .then(response => response.json())
240     .then(data => {
241         removeLoadingMessage(loadingId);
242         if (data.success) {
243             const messageId = addMessageToChat('assistant', data.response);
244             addFeedbackToMessage(messageId, 'question', question, data.response);
245         } else {
246             addMessageToChat('system', 'Error: ' + data.error);
247         }
248     })
249     .catch(error => {
250         removeLoadingMessage(loadingId);
251         console.error('Error:', error);
252         addMessageToChat('system', 'Error communicating with the server. Please
try again.');
```

```

253     });
254 }
255
256 function summarizeText() {
257     const textInput = document.getElementById('textToSummarize');
258     const text = textInput.value.trim();
259     if (!text) {
260         alert('Please enter some text to summarize.');
```

```

261         return;
262     }
263     document.getElementById('summaryResult').style.display = 'none';
264     const summarizeBtn = document.getElementById('summarizeBtn');
265     const originalBtnText = summarizeBtn.innerHTML;
266     summarizeBtn.innerHTML = '<span class="spinner-border spinner-border-sm"
role="status" aria-hidden="true"></span> Summarizing...';
267     summarizeBtn.disabled = true;
268     fetch('/generate', {
269         method: 'POST',
270         headers: { 'Content-Type': 'application/json' },
271         body: JSON.stringify({ prompt: text, function_type: 'summary' })
272     })
273     .then(response => response.json())
274     .then(data => {
275         summarizeBtn.innerHTML = originalBtnText;
276         summarizeBtn.disabled = false;
277         if (data.success) {
278             document.getElementById('summaryText').innerHTML =
marked.parse(sanitizeResponse(data.response));
279             document.getElementById('summaryResult').style.display = 'block';

```

```

280         const feedbackContainer = document.getElementById('summaryFeedback');
281         feedbackContainer.innerHTML = createFeedbackHTML('summary', text,
data.response);
282     } else {
283         alert('Error: ' + data.error);
284     }
285 })
286 .catch(error => {
287     summarizeBtn.innerHTML = originalBtnText;
288     summarizeBtn.disabled = false;
289     console.error('Error:', error);
290     alert('Error communicating with the server. Please try again.');
```

```

291 });
292 }
293
294 function generateCreative() {
295     const contentType = document.getElementById('contentType').value;
296     const topic = document.getElementById('contentTopic').value.trim();
297     if (!topic) {
298         alert('Please enter a topic or theme.');
```

```

299     return;
300 }
301 document.getElementById('creativeResult').style.display = 'none';
302 const generateBtn = document.getElementById('generateCreativeBtn');
303 const originalBtnText = generateBtn.innerHTML;
304 generateBtn.innerHTML = '<span class="spinner-border spinner-border-sm"
role="status" aria-hidden="true"></span> Generating...';
305 generateBtn.disabled = true;
306 const prompt = `Create an original ${contentType} with the theme: ${topic}. Be
imaginative and engaging.`;
307 fetch('/generate', {
308     method: 'POST',
309     headers: { 'Content-Type': 'application/json' },
310     body: JSON.stringify({ prompt: prompt, function_type: 'creative' })
311 })
312 .then(response => response.json())
313 .then(data => {
314     generateBtn.innerHTML = originalBtnText;
315     generateBtn.disabled = false;
316     if (data.success) {
317         document.getElementById('creativeTitle').textContent = `Your
${contentType.charAt(0).toUpperCase() + contentType.slice(1)}:`;
318         document.getElementById('creativeText').innerHTML =
marked.parse(sanitizeResponse(data.response));
319         document.getElementById('creativeResult').style.display = 'block';
320         const feedbackContainer = document.getElementById('creativeFeedback');
321         feedbackContainer.innerHTML = createFeedbackHTML('creative', prompt,
data.response);
322     } else {
323         alert('Error: ' + data.error);
324     }
325 })
326 .catch(error => {
327     generateBtn.innerHTML = originalBtnText;
```

```

328         generateBtn.disabled = false;
329         console.error('Error:', error);
330         alert('Error communicating with the server. Please try again.');
```

331 });

332 }

333

334 function **addMessageToChat**(sender, message) {

335 const chatContainer = document.getElementById('chatMessages');

336 const messageDiv = document.createElement('div');

337 const messageId = 'msg-' + Date.now();

338 messageDiv.id = messageId;

339 if (sender === 'user') {

340 messageDiv.className = 'message user-message';

341 messageDiv.textContent = message;

342 } else if (sender === 'assistant') {

343 messageDiv.className = 'message assistant-message';

344 messageDiv.innerHTML = marked.parse(**sanitizeResponse**(message));

345 } else {

346 messageDiv.className = 'message system-message text-danger';

347 messageDiv.textContent = message;

348 }

349 chatContainer.appendChild(messageDiv);

350 chatContainer.scrollTop = chatContainer.scrollHeight;

351 return messageId;

352 }

353

354 function **addLoadingMessage**() {

355 const chatContainer = document.getElementById('chatMessages');

356 const loadingDiv = document.createElement('div');

357 const loadingId = 'loading-' + Date.now();

358 loadingDiv.id = loadingId;

359 loadingDiv.className = 'message assistant-message';

360 loadingDiv.innerHTML = '<div class="loading-spinner"></div> Thinking...';

361 chatContainer.appendChild(loadingDiv);

362 chatContainer.scrollTop = chatContainer.scrollHeight;

363 return loadingId;

364 }

365

366 function **removeLoadingMessage**(loadingId) {

367 const loadingDiv = document.getElementById(loadingId);

368 if (loadingDiv) loadingDiv.remove();

369 }

370

371 function **addFeedbackToMessage**(messageId, functionType, prompt, response) {

372 const messageDiv = document.getElementById(messageId);

373 if (!messageDiv) return;

374 const feedbackDiv = document.createElement('div');

375 feedbackDiv.className = 'mt-2 text-center';

376 feedbackDiv.innerHTML = **createFeedbackHTML**(functionType, prompt, response);

377 messageDiv.appendChild(feedbackDiv);

378 }

379

380 function **createFeedbackHTML**(functionType, prompt, response) {

381 const feedbackId = 'feedback-' + Date.now();

```

382         return `
383             <div class="feedback-container" id="${feedbackId}">
384                 <p class="small text-muted mb-1">Was this response helpful?</p>
385                 <div class="feedback-stars mb-2">
386                     <i class="far fa-star" data-rating="1"
onclick="submitFeedback('${feedbackId}', '${functionType}',
'${encodeURIComponent(prompt)}', '${encodeURIComponent(response)}', 1)"></i>
387                     <i class="far fa-star" data-rating="2"
onclick="submitFeedback('${feedbackId}', '${functionType}',
'${encodeURIComponent(prompt)}', '${encodeURIComponent(response)}', 2)"></i>
388                     <i class="far fa-star" data-rating="3"
onclick="submitFeedback('${feedbackId}', '${functionType}',
'${encodeURIComponent(prompt)}', '${encodeURIComponent(response)}', 3)"></i>
389                     <i class="far fa-star" data-rating="4"
onclick="submitFeedback('${feedbackId}', '${functionType}',
'${encodeURIComponent(prompt)}', '${encodeURIComponent(response)}', 4)"></i>
390                     <i class="far fa-star" data-rating="5"
onclick="submitFeedback('${feedbackId}', '${functionType}',
'${encodeURIComponent(prompt)}', '${encodeURIComponent(response)}', 5)"></i>
391                 </div>
392             </div>
393         `;
394     }
395
396     function submitFeedback(feedbackId, functionType, encodedPrompt, encodedResponse,
rating) {
397         const prompt = decodeURIComponent(encodedPrompt);
398         const response = decodeURIComponent(encodedResponse);
399         const feedbackContainer = document.getElementById(feedbackId);
400         const stars = feedbackContainer.querySelectorAll('.feedback-stars i');
401         stars.forEach((star, index) => {
402             if (index < rating) {
403                 star.classList.remove('far');
404                 star.classList.add('fas');
405             } else {
406                 star.classList.remove('fas');
407                 star.classList.add('far');
408             }
409         });
410         const commentsDiv = document.createElement('div');
411         commentsDiv.className = 'mt-2';
412         commentsDiv.innerHTML = `
413             <textarea class="form-control form-control-sm" id="${feedbackId}-comments"
414                 placeholder="Any additional comments? (optional)"></textarea>
415             <button class="btn btn-sm btn-primary mt-2"
onclick="sendFeedback('${feedbackId}', '${functionType}', '${encodedPrompt}',
'${encodedResponse}', ${rating})">
416                 Submit Feedback
417             </button>
418         `;
419         feedbackContainer.appendChild(commentsDiv);
420     }
421
422     function sendFeedback(feedbackId, functionType, encodedPrompt, encodedResponse,
rating) {

```

```

423     const prompt = decodeURIComponent(encodedPrompt);
424     const response = decodeURIComponent(encodedResponse);
425     const comments = document.getElementById(`${feedbackId}-comments`).value;
426     const feedbackContainer = document.getElementById(feedbackId);
427     fetch('/feedback', {
428         method: 'POST',
429         headers: { 'Content-Type': 'application/json' },
430         body: JSON.stringify({
431             function_type: functionType,
432             prompt: prompt,
433             response: response,
434             rating: rating,
435             comments: comments
436         })
437     })
438     .then(response => response.json())
439     .then(data => {
440         if (data.success) {
441             feedbackContainer.innerHTML = `
442                 <div class="alert alert-success py-2">
443                     <i class="fas fa-check-circle"></i> Thank you for your
feedback!
444                 </div>
445             `;
446         } else {
447             feedbackContainer.innerHTML = `
448                 <div class="alert alert-danger py-2">
449                     <i class="fas fa-exclamation-circle"></i> Error saving
feedback. Please try again.
450                 </div>
451             `;
452         }
453     })
454     .catch(error => {
455         console.error('Error:', error);
456         feedbackContainer.innerHTML = `
457             <div class="alert alert-danger py-2">
458                 <i class="fas fa-exclamation-circle"></i> Error saving feedback.
Please try again.
459             </div>
460         `;
461     });
462 }

463
464 function loadHistory() {
465     fetch('/history')
466     .then(response => response.json())
467     .then(data => {
468         const historyContainer = document.getElementById('historyList');
469         const emptyHistory = document.getElementById('emptyHistory');
470         if (data.success && data.history && data.history.length > 0) {
471             emptyHistory.style.display = 'none';
472             historyContainer.innerHTML = '';
473             data.history.forEach((item, index) => {

```

```

474         const historyItem = document.createElement('div');
475         historyItem.className = 'card mb-3';
476         historyItem.innerHTML = `
477             <div class="card-header d-flex justify-content-between
align-items-center">
478                 <span>
479                     <i class="fas ${getFunctionTypeIcon(item.type)}
me-2"></i>
480                     ${formatFunctionType(item.type)}
481                 </span>
482                 <small class="text-muted">${item.timestamp}</small>
483             </div>
484             <div class="card-body">
485                 <h6 class="card-subtitle mb-2 text-muted">Prompt:</h6>
486                 <p class="card-text">${item.prompt}</p>
487                 <h6 class="card-subtitle mb-2 text-muted">Response:
</h6>
488                 <div class="card-
text">${marked.parse(sanitizeResponse(item.response))}</div>
489             </div>
490         `;
491         historyContainer.appendChild(historyItem);
492     });
493     } else {
494         emptyHistory.style.display = 'block';
495         historyContainer.innerHTML = '';
496     }
497 })
498 .catch(error => {
499     console.error('Error:', error);
500     document.getElementById('historyList').innerHTML = `
501         <div class="alert alert-danger">
502             Error loading history. Please try again.
503         </div>
504     `;
505 });
506 }
507
508 function clearHistory() {
509     if (!confirm('Are you sure you want to clear your conversation history?'))
return;
510     fetch('/clear_history', { method: 'POST' })
511     .then(response => response.json())
512     .then(data => {
513         if (data.success) {
514             loadHistory();
515         } else {
516             alert('Error: ' + data.error);
517         }
518     })
519     .catch(error => {
520         console.error('Error:', error);
521         alert('Error clearing history. Please try again.');
```

```
523     }
524
525     function downloadHistory() {
526         window.location.href = '/download_history';
527     }
528
529     function getFunctionTypeIcon(type) {
530         switch(type) {
531             case 'question': return 'fa-question-circle';
532             case 'summary': return 'fa-file-alt';
533             case 'creative': return 'fa-paint-brush';
534             default: return 'fa-comment';
535         }
536     }
537
538     function formatFunctionType(type) {
539         switch(type) {
540             case 'question': return 'Question & Answer';
541             case 'summary': return 'Text Summary';
542             case 'creative': return 'Creative Content';
543             default: return type.charAt(0).toUpperCase() + type.slice(1);
544         }
545     }
546 </script>
547 </body>
548 </html>
```